Introduction to the Yale SOM HPC

Abstract. This guide introduces the Yale SOM "High Performance Computing" cluster— a shared computing environment for SOM researchers. The cluster (or "the HPC") is a great place to get work done that doesn't fit on your laptop. This guide explains how to use the HPC skillfully whilst simultaneously being a good steward of this shared and lightly regulated resource. There is a companion set of Claude Code skills for AI agents that contains more details. If your AI agents are aiding you in your HPC work, be sure to load those skills.

Contents

What is an HPC, and why use one? . . . 2
The mental model: a shared computer . . . 2
Login nodes vs compute nodes . . . 2
Connecting to the cluster . . . 3
The job scheduler: thinking in jobs, not processes . . . 4
Right-sizing: the post-job feedback loop . . . 5
The filesystem zoo . . . 6
Moving files in and out of the HPC . . . 8
Software: modules, environments, and no sudo . . . 9
Running Python . . . 9
Running R . . . 11
Running Stata . . . 12
GPUs . . . 13
Working with large data . . . 14
Acquiring data: WRDS, APIs, scraping . . . 15
Starting a new project . . . 17
HPC etiquette . . . 17
Working with Claude Code on the HPC . . . 18
Getting help . . . 19
Appendix: example files . . . 20

What is an HPC, and why use one?

The Yale SOM HPC is where you send work that has outgrown your laptop. The price of that extra capacity is that you now launch work through a shared scheduler instead of running directly, at will, on hardware you own.

The HPC is in the basement of Evans Hall roughly under Zhang Auditorium. It looks like a bunch of pizza-box-shaped servers/computers, stacked in racks, connected with cables. Most of those servers are *compute nodes*; one server acts as a *login node* or *head node*. All these nodes share a fast filesystem and are managed by a *job scheduler* that allocates CPU cores, memory, GPUs, and time across many users. The Yale SOM cluster lives at `hpc.som.yale.edu` on the internet, but you can only access it from the campus network or Yale VPN.

You want to use the cluster when your laptop has become the wrong tool: the data no longer fits in RAM; programs take too long to babysit; your script needs to run hundreds of times with different parameters; the work needs a GPU you do not own; or you need to share some "biggish" data with SOM colleagues. You should not move a 30-second script to the cluster just because the cluster exists. Indeed, you'll be unhappy with the outcome! On your laptop/desktop, you're the boss and you don't need to think about **how** to run a job or be inconvenienced by other users. You should only move to the HPC if you have to.

This guide has two jobs. First, it gives the mental model for a shared, scheduled, multi-tenant system. Most cluster rules follow from that model. Second, it gives the recipes you will reuse: Slurm scripts, module loads, file-transfer patterns, language-specific workflows, and post-job checks.

The rest of the guide moves from concepts to practice. Sections 2–4 cover the shared-instrument model, login versus compute nodes, and secure connections. Sections 5–7 cover Slurm, resource right-sizing, and the filesystem. Sections 8–9 cover file movement and software installation. Sections 10–12 cover Python, R, and Stata. Sections 13–15 cover GPUs, larger-than-memory data, and data acquisition. Sections 16–17 cover project setup and etiquette. Section 18 explains how this guide pairs with the Claude Code marketplace. Section 19 collects the commands and contacts to keep nearby, and the appendix gives a complete first-job example.

The mental model: a shared computer

The cluster is shared environment: there's a bunch of us on there, we all want to get work done, and we need to be considerate to each other if that is going to happen. Our resource scheduling software—Slurm—helps us in this regard. With Slurm, each of us can say what we think we need to get our research done and Slurm will make it happen in the most efficient and fair way that it can. Of course, that often means there's a little bit of waiting to get the CPUs, GPUs, or RAM that you want.

Login nodes vs compute nodes

The login node is the front desk of the basement machine, not the workshop. Use it to check in, edit files, run `git`, browse modules with `module spider`, perform small setup steps, and submit jobs. Do not use it to run analyses, train models, load a 5 GB CSV into pandas, or

hold significant RAM. When one person runs heavy compute on the login node, everyone else's shell gets slow. It is super frustrating!

The right way to "just try something quickly" is an interactive job on a compute node:

```
srun --partition=cpunormal --cpus-per-task=2 --mem=8G --time=01:00:00 --pty bash
```

This reserves two cores, 8 GB of memory, and one hour of walltime, then puts you in a shell on a real compute node. When you are done, leave with `Ctrl+D` or `exit`. An interactive allocation left open during a meeting is still an allocation.

> Claude Code skills: `overview`, `managing-jobs`
> Claude Code knows not to run heavy commands on the login node, and reaches for `srun --pty` or `sbatch` instead. If it ever proposes running a long-running command on a login node, push back.

Connecting to the cluster

Connect to the cluster with SSH:

```
ssh <your-netid>@hpc.som.yale.edu
```

For daily work, make three small improvements. First, create an SSH key on your laptop with `ssh-keygen -t ed25519`, protect it with a passphrase, and copy only the public key to the cluster with `ssh-copy-id <netid>@hpc.som.yale.edu`. Never copy the private key (`id_ed25519`) onto GPFS. Second, use an SSH agent — `ssh-agent` on Linux or the macOS keychain on a Mac — so you type the passphrase once per session. Third, add a host block to `~/.ssh/config` so you can type `ssh hpc` instead of the full hostname:

```
Host hpc
  HostName hpc.som.yale.edu
  User <your-netid>
```

Use `ForwardAgent yes` only when you need it, usually for GitHub or another SSH hop. Agent forwarding is useful, but it is not magic: while the connection is open, the remote host can ask your laptop's agent to sign. If GitHub authentication worked yesterday and fails today inside `tmux`, do not copy keys to the cluster. Start a fresh SSH login or refresh `SSH_AUTH_SOCK`; the socket path went stale, not the private key.

You must reach the cluster from a Yale network path: campus network or Yale VPN. If you are at home and SSH just hangs, check the VPN before accusing your keys. SSH to a compute node only after Slurm has allocated that node to you; use the hostname printed inside the allocation, then tunnel to that node for Jupyter or VS Code. Notebook kernels and heavy editor extensions belong on compute nodes, not the login node.

> Claude Code skill: `connecting-securely`
> Full recipes for SSH keys, agent forwarding, stale agent sockets, Jupyter/VS Code tunnels, and compute-node SSH live in `connecting-securely`. Ask Claude Code to "set up an SSH config for the SOM HPC" and it will follow that skill.

The job scheduler: thinking in jobs, not processes

On your laptop, `python script.py` starts a process. On the cluster, you submit a *job*: a resource request plus the commands to run. Think reservation system, not magic shell. Slurm queues the job, finds a node where the request fits, and starts it there. If the job uses more memory or time than you promised, Slurm kills it. That is the contract: you estimate; Slurm enforces.

The contract has four axes — CPU cores, memory, walltime, and GPUs — declared with the directives in Table 1.

Axis	*Slurm directive*
CPU cores	`--cpus-per-task=N`
Memory	`--mem=16G` (per node) or `--mem-per-cpu=4G`
Walltime	`--time=HH:MM:SS`
GPUs	`--gres=gpu:1` or `--gpus=1` (only when needed)

Table 1: The four resource axes of a Slurm job.

The fifth knob, `--ntasks`, causes the most confusion. Use `--ntasks=N` when you need N cooperating processes, usually MPI ranks. Use `--cpus-per-task=N` when one process needs N CPU cores for threads. Most SOM jobs are not MPI jobs, so most sbatch scripts should say `--ntasks=1 --cpus-per-task=N`. Reach for `--ntasks>1` only when you know why multiple processes must cooperate.

A *partition* is a queue tied to a set of nodes. The SOM cluster currently exposes five. `default_queue` mixes CPU-only and A40 GPU nodes and caps walltime at four hours, so it is good for short tests and much interactive work. `cpunormal` is the CPU-only workhorse for longer batch jobs. `gpunormal` carries RTX 8000 and A100 GPUs. `h100` is one node with four H100 GPUs; treat it as scarce because it is. `build` exists for compiling software. Before production submissions, check the live view with `sinfo -s`.

A minimal sbatch script puts the contract in one file:

```
#!/bin/bash
#SBATCH --job-name=test
#SBATCH --partition=default_queue
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

hostname
python --version
```

The thread-environment block matters more than it looks; Section 10 explains why. Submit with `sbatch my_job.sh`, watch your jobs with `squeue --me`, inspect one job with `scontrol show job <id>`, and cancel with `scancel <id>`. The states you will see most often are `R` (running), `PD` (pending), and `CG` (completing).

If `sbatch job.sh` fails with `bad interpreter`, `: not found`, or `$'\r'`, the script has Windows CRLF line endings. Fix it with `dos2unix job.sh`, or with `sed -i 's/\r$//' job.sh` if `dos2unix` is not installed, and configure your editor to write LF for `.sh` files.

Interactive sessions are for debugging, not unattended work. Ask for the smallest allocation that lets you debug, set a short time limit, and exit when you are finished:

```
srun --partition=cpunormal --cpus-per-task=2 --mem=8G --time=01:00:00 --pty bash
```

If a job sits in `PD` for too long, ask Slurm why. Start with `squeue --me --start` for the estimated start time and queueing reason, then use `scontrol show job <id>` for the full record. The common trap is asking for resources that do not exist on any node: for example, 256 GB of RAM when the largest node has 192 GB, or 64 cores on partitions whose nodes top out at 48. Slurm may queue that job forever rather than telling you it is impossible. Match requests to real hardware with `sinfo -s` and `scontrol show node`.

After a job finishes, `sacct -j <id>` reports its history with the columns you choose:

```
sacct -j <id> --format=JobID,JobName,State,Elapsed,MaxRSS,ReqMem,AllocCPUS
```

Two Slurm patterns cover much of research computing. *Job arrays* run the same script many times with different indexes: add `#SBATCH --array=1-500%50`, where `%50` limits concurrency to 50 tasks, and read the index from `$SLURM_ARRAY_TASK_ID`. *Dependencies* express order: submit B only after A succeeds with `sbatch --dependency=afterok:<jobid-of-A> B.sh`.

> Claude Code skills: `managing-jobs`, `self-diagnosing-resource-use`
> The `managing-jobs` skill has the full pattern library — array throttling, dependency chains, partition selection, the right-sizing loop. Pair it with `self-diagnosing-resource-use` for post-mortems.

Right-sizing: the post-job feedback loop

Every resource request is a forecast. The skillful habit is to compare the forecast with what happened and make the next request tighter. Two commands do most of the work:

```
seff <job-id>
sacct -j <job-id> --format=JobID,Elapsed,MaxRSS,ReqMem,AllocCPUS,State
```

For CPU-bound work, CPU efficiency above about 50% is reasonable. Efficiency between 10% and 50% often means I/O-bound work or an over-request. Below 10% usually means waste. If CPU efficiency is low, request fewer CPUs or fix the parallelism before scaling up. For memory, set the next `--mem` to about 1.5-2× the observed peak `MaxRSS`, not 10×. If you requested 128 GB and used 4 GB, Slurm reserved 124 GB for you that your code never touched. For walltime, extrapolate from a sample run. If the work is resumable, prefer 1-4 hour chunks that can backfill into idle slots over one multi-day job.

Treat future caps as if they already exist. SOM HPC is still lightly enforced compared with many clusters, but shared systems naturally move toward hard per-user CPU, memory, GPU, or interactive limits. Right-sized work stays schedulable when that happens.

Over-requesting is not harmless. It makes you wait longer because Slurm must find a larger empty slot, and it withholds idle resources from people behind you. Inspect every serious job; adjust the next submission. The habit pays off more reliably than any one clever optimization.

> Claude Code skill: `self-diagnosing-resource-use`
> Ask Claude Code "did my last job use what it asked for?" and it will run `sacct` and `seff`, interpret the output, and propose a tighter request for next time.

The filesystem zoo

The cluster has several filesystems because one storage policy cannot serve every job. Pick the right one before you start. Otherwise you will fill `$HOME`, punish shared metadata, or lose intermediates when a node reboots. The basement machine has more than one closet; do not put everything in the coat closet.

Path	*What it is for*	*Lifetime*
`$HOME` (= `/home/$USER`, same as `/gpfs/home/$USER`)	Code, configs, dotfiles, small things. Backed up.	Permanent
`/gpfs/project/<proj>/`	Shared team data, code, outputs. Request via `somit@yale.edu`.	Project lifetime
`/gpfs/scratch60/$USER/`	Large temporary working files; staging area.	Treat as ephemeral; clean it
`/tmp` on a compute node	Local fast scratch, just for this job.	Gone when the job ends

Table 2: The four filesystems you will use, and what each is for.

The short version: code lives in `$HOME` or in `code/` under project space; data lives under `/gpfs/project/<proj>/data/` or `/gpfs/scratch60/$USER/`. `$HOME` is small and is the wrong place for 200 GB of intermediates. `/gpfs/scratch60/$USER` may not exist until you create it with `mkdir -p /gpfs/scratch60/$USER`. Scratch is a motel, not a museum: use it, then clean it.

The metadata-storm warning

GPFS is fast at streaming large files and bad at being handed millions of tiny chores. Every file creation, directory listing, and `stat` call touches a metadata server shared by the cluster. A job that writes one file per row of a 2-million-row dataset can make `ls` slow for everyone.

Use three counter-patterns. Write Parquet instead of per-row CSVs. Tar or zip directories of small files when you are done with them. For many small intermediates inside one job, write them to the compute node's `/tmp`, then copy one tarball or final output back to GPFS.

Atomic writes and resumable outputs

Jobs can die halfway through a write: memory limit, walltime, preemption, maintenance, or plain bugs. If the script writes directly to `output.parquet`, a half-written file may look finished. Write to a temporary path on the same filesystem, then rename:

```
import os
tmp = "output.parquet.tmp"
df.write_parquet(tmp)
os.replace(tmp, "output.parquet")  # atomic on the same filesystem
```

Pair atomic writes with skip-if-exists at the top of each unit of work. Then a killed job can be rerun and will pick up where it left off. For arrays, the usual shape is one output per task — for example, `output/task_0001.parquet`, `output/task_0002.parquet` — followed by a final combine step. That is much better than thousands of per-row CSVs and safer than many tasks mutating one large file in place.

For high-I/O work inside one job, stage onto compute-node `/tmp` and copy only final outputs back to GPFS:

```
workdir=$(mktemp -d "${TMPDIR:-/tmp}/job_${SLURM_JOB_ID:-local}.XXXXXX")
export TMPDIR="$workdir"
trap 'rm -rf "$workdir"' EXIT

cp /gpfs/project/<proj>/data/input.parquet "$workdir"/
srun .venv/bin/python src/process.py \
  --input "$workdir/input.parquet" \
  --output "$workdir/output.parquet" &
job_pid=$!
wait "$job_pid"
cp "$workdir/output.parquet" /gpfs/project/<proj>/output/
```

The `cmd & wait $!` shape lets bash run the cleanup trap when Slurm sends `SIGTERM` near the time limit. `SIGKILL` still bypasses traps, which is why `/tmp` is only scratch.

> Claude Code skills: `using-the-filesystem`, `working-with-large-data`
> `using-the-filesystem` has the full pattern library; `working-with-large-data` covers the Parquet / DuckDB / Polars patterns that avoid both metadata storms and out-of-memory kills.

Moving files in and out of the HPC

A new user's first obstacle is often not Slurm. It is getting code and data onto the cluster, then getting results back off. Most cases fit one of the patterns below.

I already have a codebase locally

Use Git for code. Do not hand-carry a codebase to the basement with `scp` if Git can do the job cleanly. Push the repository to GitHub or Yale's GitHub Enterprise, then clone it in project space:

```
cd /gpfs/project/<proj>/
git clone git@github.com:<you>/<repo>.git code
```

From then on, edit wherever you work — laptop, cluster shell, VS Code Remote-SSH — and use `git pull` and `git push` to synchronize. This gives you two useful things for free. First, your code has an off-cluster backup; if the cluster has a bad day, GitHub still has the repository. Second, you get history for the inevitable "what changed last week?" Commit before each serious run, print the commit hash into the job log, and use `git bisect` when a result stops reproducing.

I have a dataset on my laptop

For one-shot transfers, `scp` is fine:

```
scp local.csv somhpc:/gpfs/project/<proj>/data/
```

For anything large, slow, or interruption-prone, use `rsync` because it can resume:

```
rsync -avP --partial local_dir/ somhpc:/gpfs/project/<proj>/data/
```

Do not check data into Git. That is what `/gpfs/project/` is for. Git LFS exists, but it is rarely worth the trouble for SOM workflows.

I have results on the cluster I want back

Use the same tools in the other direction:

```
rsync -avP --partial somhpc:/gpfs/project/<proj>/output/ ./output/
```

VS Code Remote-SSH can fetch individual files transparently when you open them. The pattern to avoid is pulling thousands of small output files one by one. Tar them on the cluster first, both to spare GPFS metadata and to make the transfer one fast operation instead of thousands of slow ones:

```
tar -czf results.tar.gz output/
```

then `scp` the one tarball.

I want to fetch from a public URL

Use `wget` or `curl` to write directly into `/gpfs/project/`, preferably from a compute node so the login node is not occupied. The cluster's outbound IP is shared, so credentialed downloads, APIs, and rate-limited services need the care described in Section 15.

What does not belong on the HPC

Final PDFs, slide decks, Overleaf projects, and manuscript drafts belong on your laptop, in Overleaf, or in Dropbox. The cluster is for compute and the data feeding it. Build the figure on the cluster; copy it out before putting it in the paper.

> Claude Code skills: `connecting-securely`, `using-the-filesystem`, `starting-a-new-project`
> Ask Claude Code to "clone my GitHub repo into project space and set up a uv environment" and it will combine these three skills.

Software: modules, environments, and no sudo

On the cluster, you do not have `sudo`. The basement servers are not your MacBook with a bigger fan. You cannot solve software problems with `apt install`, `brew install`, or `pip install --system`. Use three layers instead, in this order: cluster-managed modules, project-local environments, and containers.

The first layer is *modules*, served by Lmod and built behind the scenes with Spack. Search with `module spider <name>`, load with `module load <name>`, inspect the current environment with `module list`, and clear inherited state with `module purge`. Git, R, Stata, CUDA, MATLAB, Apptainer, and sometimes Python are module-provided. Load required modules inside scripts; do not rely on whatever happened to be loaded in your login shell.

The second layer is *project-local environments*. For Python, the current default is `uv`: it is fast on GPFS, creates a `.venv/`, and writes a `uv.lock` file that reproduces the environment. Avoid `pip install --user`, avoid installs inside jobs or job arrays, and do not share one `$HOME`-level environment across unrelated projects. For R, use `renv` per project and restore once during setup, not inside hundreds of jobs.

The third layer is *containers*, via Apptainer, for cases modules and user-space environments cannot handle: old GLIBC errors, complex C/CUDA dependencies, or a published Docker image you need to reproduce. Load it with `module load apptainer`. For small user tools under `~/.local/bin`, prefer static or musl Linux binaries when available; they avoid many `GLIBC_2.xx not found` failures.

> Claude Code skills: `installing-software`, `running-python`, `running-r`
> `installing-software` is the umbrella skill; the language-specific ones layer on top. Ask Claude Code to "install package X" and it will pick the right layer (module, uv, renv, Apptainer) for you.

Running Python

For new Python projects, use `uv` and keep the environment inside the project code directory. Treat it as part of the project, not as a mysterious pet living in `$HOME`:

```
cd /gpfs/project/<proj>/code
uv init --app
uv add polars pyarrow duckdb
uv sync --frozen
```

Commit `pyproject.toml` and `uv.lock`. Do not commit `.venv/`. Run `uv sync --frozen` during setup on the login node, not inside Slurm jobs or arrays. Mutating environments in flight is slow, hard to reproduce, and a good way to make GPFS do bookkeeping instead of useful work. Plain `pip` is not forbidden; `pip install --user` and per-job installs are the bad patterns.

A typical Python sbatch script looks like this. Notice the thread exports and the `srun .venv/bin/python` launch line:

```bash
#!/bin/bash
#SBATCH --job-name=py_analysis
#SBATCH --partition=default_queue
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

cd /gpfs/project/<proj>/code
srun .venv/bin/python src/main.py
```

Use `srun .venv/bin/python ...` for long or resumable jobs so Slurm signals reach Python. For short exploratory jobs where signal-based shutdown does not matter, `uv run python src/main.py` is acceptable. In Python, read Slurm values with fallbacks so the same code works locally and on the cluster:

```python
import os
n_cpus = int(os.environ.get("SLURM_CPUS_PER_TASK", "0")) or os.cpu_count() or 1
job_id = os.environ.get("SLURM_JOB_ID", "local")
```

Why the thread-environment block matters

NumPy, SciPy, scikit-learn, Polars, NumExpr, and much of the scientific Python stack call threaded native libraries. By default, those libraries may spawn one thread per CPU they see on the *node*, not per CPU you requested from Slurm. The `*_NUM_THREADS` exports pin them to the cores Slurm gave you. The `${SLURM_CPUS_PER_TASK:-1}` form matters because a bare `$SLURM_CPUS_PER_TASK` is empty outside Slurm.

Python data defaults and resumable outputs

For tabular work, try Polars lazy scans and DuckDB before multiprocessing. Store reusable data as compressed Parquet, convert to pandas only at library boundaries, and write one Parquet file per array task or chunk instead of mutating one large file in place.

Write each unit of work so it skips existing outputs, writes to `<name>.tmp`, and renames only after success:

```python
from pathlib import Path
import polars as pl

output = Path("/gpfs/project/<proj>/output/task_0001.parquet")
if output.exists():
    raise SystemExit(0)

tmp = output.with_suffix(".parquet.tmp")
pl.DataFrame({"ok": [1]}).write_parquet(tmp)
tmp.rename(output)
```

> Claude Code skills: `running-python`, `parallel-python`, `accelerating-python`
> Three layered skills. `running-python` covers uv, Slurm, logging, and resumable outputs. `accelerating-python` covers DuckDB, Polars, and Numba before parallelism. `parallel-python` covers worker sizing, arrays, and signal handling once you actually need workers.

Running R

Load R through the module system, and load it explicitly in job scripts. Your interactive shell may remember what you loaded yesterday; a batch job does not owe you that favor:

```
module spider r
module load r
R --version
```

Use `renv` for project-local package management. Initialize and restore on the login node during setup, commit `renv.lock` and `.Rprofile`, and do not commit `renv/library/`:

```
install.packages("renv")
renv::init()
renv::install(c("data.table", "arrow", "fixest"))
renv::snapshot()
```

For shared projects, put the renv library under project space by adding this to `.Rprofile`:

```
Sys.setenv(RENV_PATHS_LIBRARY = "/gpfs/project/<proj>/environments/renv/library")
```

Run `renv::restore()` once during setup, not inside a Slurm array. A typical R sbatch script looks like this:

```
#!/bin/bash
#SBATCH --job-name=r_analysis
#SBATCH --partition=default_queue
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

module purge
module load r

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}

cd /gpfs/project/<proj>/code
srun Rscript src/main.R
```

Inside R, match package threads to the Slurm allocation:

```
slurm_cpus <- Sys.getenv("SLURM_CPUS_PER_TASK", "")
n_cpus <- if (nzchar(slurm_cpus)) as.integer(slurm_cpus) else parallel::detectCores()
data.table::setDTthreads(n_cpus)
```

For ordinary research code, tidyverse is readable and shareable. Switch to `data.table` or `dtplyr` when measured memory or runtime says you should. Use Arrow and Parquet for reusable data on GPFS.

> Claude Code skill: `running-r`
> `running-r` covers `renv` setup, batch invocation, and the BLAS, OpenMP, `srun Rscript`, and `data.table` thread-control checklist.

Running Stata

Run Stata on the cluster in batch mode. Put temporary files on scratch, match Stata/MP processors to the Slurm CPU request, and close idle sessions because Stata licenses are shared. A Stata license left idle is still a seat someone else cannot use.

```bash
#!/bin/bash
#SBATCH --job-name=stata_run
#SBATCH --partition=default_queue
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

module purge
module load stata

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export STATATMP=/gpfs/scratch60/$USER/stata-tmp/${SLURM_JOB_ID}

mkdir -p "$STATATMP" logs
trap 'rm -rf "$STATATMP"' EXIT

cd /gpfs/project/<proj>/code
stata-mp -b do src/main.do
```

Use a do-file preamble like this:

```
capture log close _all
log using "logs/main_${SLURM_JOB_ID}.log", replace text

local ncpus : env SLURM_CPUS_PER_TASK
if "`ncpus'" == "" local ncpus 1
set processors `ncpus'

local statatmp : env STATATMP
di "STATATMP=`statatmp'"

set more off
```

Each Stata array task should write a separate output file. Use `compress`, drop unneeded variables before merges, and keep `tempfile` intermediates out of `$HOME`.

> Claude Code skill: `running-stata`
> `running-stata` covers batch invocation, log handling, `STATATMP`, MP-core sizing, and the license-pool etiquette.

GPUs

Request a GPU only when the code actually uses CUDA: PyTorch, JAX, TensorFlow, RAPIDS, CuPy, or your own CUDA kernel. A GPU assigned to ordinary NumPy, Stata, dataframe work, downloading, tokenization, or scraping is a very expensive space heater in the basement, not acceleration.

Start with one GPU:

```bash
#!/bin/bash
#SBATCH --job-name=train
#SBATCH --partition=gpunormal
#SBATCH --gres=gpu:1
#SBATCH --time=01:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

cd /gpfs/project/<proj>/code
nvidia-smi
nvidia-smi --query-
gpu=timestamp,utilization.gpu,utilization.memory,memory.used,power.draw \
  --format=csv -l 10 > logs/gpu_${SLURM_JOB_ID}.csv &
srun .venv/bin/python train.py
```

Request multiple GPUs only when the code explicitly uses multiple GPUs. Split CPU preprocessing from GPU training: run download, cleaning, tokenization, and feature construction as a CPU job, then submit the GPU job with `--dependency=afterok:<cpu-jobid>`. If `GPU-Util` stays near 0%, VRAM is 0 MB, or utilization alternates between idle and saturated, cancel and debug before spending more GPU-hours.

Current rough guide, to be verified with `sinfo`: `gpunormal` contains RTX 8000/A100-class GPUs; A100 nodes hold three GPUs. RTX 8000/A40 are 48 GB nominal, with RTX 8000 reporting roughly 46 GB usable. The `h100` partition is one node with four 80 GB H100s and is the scarcest resource on the cluster. GPU nodes currently support the CUDA 12.8 runtime, so PyTorch/JAX wheels with bundled CUDA often work without `module load cuda`; load a CUDA module when you need `nvcc` or a specific toolkit:

```bash
module spider cuda
module load cuda
```

`nvidia-smi` works inside a GPU allocation, not on the login node. At the start of training, also check `torch.cuda.is_available()` or the equivalent for your framework.

The cardinal sin: idle interactive GPUs

Do not allocate an interactive GPU and then go to lunch, a meeting, or bed. Cancel interactive GPU sessions the moment you stop typing: `exit` from inside, or `scancel <job-id>` from outside.

> Claude Code skill: `using-gpus`
> `using-gpus` walks through GPU partition selection, monitoring, CUDA module choices, preprocessing splits, OOM handling, and the idle-GPU detection patterns Claude Code uses to flag wasted allocations.

Working with large data

When data fits comfortably in RAM, simple tools are fine. When it does not, query before loading and store reusable data in columnar formats. Do not ask Python to swallow the whole dataset just to count a few rows. Parquet is smaller and faster than CSV, supports column pruning and predicate pushdown, and avoids metadata storms. DuckDB, Polars lazy scans, and Arrow can stream over datasets larger than the job's RAM.

Inspect files before writing a large script:

```
duckdb -c "DESCRIBE SELECT * FROM 'data.csv'"
duckdb -c "SELECT * FROM 'data.csv' LIMIT 10"
duckdb -c "COPY (SELECT * FROM 'data.csv') TO 'data.parquet' (FORMAT PARQUET)"
```

DuckDB example:

```
import duckdb
con = duckdb.connect()
con.execute("""
COPY (
  SELECT firm_id, year, AVG(ret) AS mean_ret
  FROM read_parquet('/gpfs/project/<proj>/data/returns/*.parquet')
  WHERE year BETWEEN 1990 AND 2020
  GROUP BY firm_id, year
) TO '/gpfs/project/<proj>/output/mean_ret.parquet' (FORMAT PARQUET)
""")
```

Polars lazy example:

```
import polars as pl

result = (
    pl.scan_parquet("/gpfs/project/<proj>/data/panel/*.parquet")
    .filter(pl.col("fyear") >= 2010)
    .select(["gvkey", "fyear", "assets", "sales"])
    .group_by("fyear")
    .agg(pl.col("sales").mean())
)
result.sink_parquet("/gpfs/project/<proj>/output/sales_by_year.parquet")
```

R Arrow example:

```
library(arrow)
library(dplyr)
```

```r
ds <- open_dataset("/gpfs/project/<proj>/data/panel")
result <- ds |>
  filter(fyear >= 2010) |>
  select(gvkey, fyear, assets, sales) |>
  group_by(fyear) |>
  summarise(mean_sales = mean(sales, na.rm = TRUE)) |>
  collect()
write_parquet(result, "/gpfs/project/<proj>/output/sales_by_year.parquet")
```

Use SQLite with WAL for small local caches and lookup tables. Use one connection per process and keep writes single-writer:

```python
import sqlite3
conn = sqlite3.connect("/gpfs/project/<proj>/cache/lookup.db")
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("CREATE TABLE IF NOT EXISTS cache (key TEXT PRIMARY KEY, value TEXT)")
conn.commit()
```

Always sample first. A 10,000-row sample is enough to debug code, estimate memory, and check column names. For arrays, write one Parquet file per task and combine later with `pl.scan_parquet("output/task_*.parquet")` or DuckDB.

> Claude Code skills: `working-with-large-data`, `accelerating-python`
> `working-with-large-data` covers Parquet conversion, DuckDB, Polars, Arrow, SQLite caches, sample-first workflows, and chunked pipelines. `accelerating-python` covers when to add Numba or parallelism *after* you have fixed the data layer.

Acquiring data: WRDS, APIs, scraping

Downloading data from the cluster changes the blast radius. From the outside world, many jobs may look like one very busy machine in Evans Hall. Every job shares one outbound IP, repeated downloads waste time and money, and credentials must never land in scripts or Git.

For WRDS or Postgres, keep connection details in `~/.pg_service.conf` and secrets in `~/.pgpass`, both `chmod 600`, then connect by service name:

```
# ~/.pg_service.conf
[wrds]
host=wrds-pgdata.wharton.upenn.edu
port=9737
dbname=wrds
user=yourwrdsid
```

```python
import psycopg
with psycopg.connect("service=wrds") as conn, conn.cursor() as cur:
    cur.execute("select permno, date, ret from crsp.msf where date >= %s",
("2010-01-01",))
    rows = cur.fetchall()
```

Do not run the same WRDS extract inside every analysis job. Download once to project storage, keep the raw extract, and analyze local Parquet files. When parallel workers share a database, use `psycopg_pool.ConnectionPool`, create pools inside worker processes, and set `max_size` deliberately. Naive parallelism can exceed WRDS or Postgres connection limits.

```python
from psycopg_pool import ConnectionPool
```

```python
# Create inside the worker process if using multiprocessing.
pool = ConnectionPool("service=wrds", min_size=1, max_size=4)

def fetch_permno(permno):
    with pool.connection() as conn, conn.cursor() as cur:
        cur.execute("select date, ret from crsp.msf where permno = %s", (permno,))
        return cur.fetchall()
```

Cache paid API calls, web pages, and slow endpoints by a hash of the request payload, and write the cache atomically:

```python
import hashlib, json
from pathlib import Path

CACHE = Path("/gpfs/project/<proj>/cache/api")
CACHE.mkdir(parents=True, exist_ok=True)

def cache_key(payload: dict) -> str:
    return hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()

def cached_call(payload: dict):
    path = CACHE / f"{cache_key(payload)}.json"
    if path.exists():
        return json.loads(path.read_text())
    response = call_expensive_api(payload)
    tmp = path.with_suffix(".json.tmp")
    tmp.write_text(json.dumps(response))
    tmp.rename(path)
    return response
```

Respect rate limits and `robots.txt`, add retries with exponential backoff, and add deliberate sleeps when scraping:

```python
import time, requests

session = requests.Session()

def fetch(url, attempts=5):
    for attempt in range(attempts):
        r = session.get(url, timeout=30)
        if r.status_code == 429 and attempt < attempts - 1:
            time.sleep(int(r.headers.get("Retry-After", "0") or "0") or 2 ** attempt)
            continue
        r.raise_for_status()
        time.sleep(1.0)  # deliberate politeness delay for scraping
        return r
```

For paid APIs or LLM calls, enforce a cost cap in code before submitting a long job:

```python
MAX_BUDGET_DOLLARS = 50.0
spent = 0.0
for request in requests_to_make:
    if spent >= MAX_BUDGET_DOLLARS:
        raise RuntimeError(f"budget exceeded: ${spent:.2f}")
    result = cached_call(request)
    spent += estimate_cost(result)
```

Store raw HTML or JSON responses before parsing. Then a parser change does not force another download.

> Claude Code skill: `acquiring-data`
> `acquiring-data` covers WRDS service files, connection pooling, API caches, rate limits, retries, cost caps, scraping etiquette, and the credentials pattern in detail.

Starting a new project

A predictable project layout under `/gpfs/project/<proj>/` saves future you from archaeology. Six months from now, future you will not remember where the "final final" CSV went. The recommended skeleton is:

```
/gpfs/project/myproj/
├── code/                    # Git repo, pushed to GitHub
│   ├── README.md
│   ├── pyproject.toml
│   ├── uv.lock
│   ├── renv.lock
│   ├── src/
│   ├── scripts/
│   └── slurm/
├── data/
│   ├── raw/                 # original inputs, read-only after ingest
│   └── derived/             # rebuildable intermediates
├── output/                  # results, regeneratable from code+data
├── logs/                    # Slurm logs (%x_%j.out)
├── cache/                   # request-hash caches
└── environments/            # project-local env state not committed
```

Git tracks `code/` and lockfiles, not data, outputs, logs, caches, or environments. A minimal `.gitignore` excludes `.venv/`, `renv/library/`, `data/`, `output/`, `logs/`, `cache/`, `.env`, `*.out`, and `*.err`. The README should say what the project does, where raw data comes from, how to rebuild outputs, and which Slurm script is the first test job. A thin `Justfile` or `Makefile` with targets such as `setup`, `test`, `submit-test`, and `clean-scratch` is often enough.

After ingest, make raw data read-only by convention or permission. Write modified data to `data/derived/`. Outputs should be regeneratable: deleting `output/` and rerunning the pipeline should rebuild them. Use one Python or R environment per project rather than a shared `$HOME` environment.

> Claude Code skill: `starting-a-new-project`
> Ask Claude Code to "set up a new project under `/gpfs/project/`" and it will create this layout, initialize Git and uv, write ignores and a README skeleton, and add a starter sbatch script.

HPC etiquette

The HPC is shared, and `squeue` is public. Poor cluster citizenship is not abstract; it shows up as colleagues waiting behind resources you are holding. On a small cluster, the room may be in the basement, but the queue is very visible.

The most expensive bad habit is idle allocation: an interactive session left open overnight, a 24-hour batch request for a 30-minute task, or an interactive GPU abandoned during a meeting. Cancel idle sessions with `scancel <job-id>`. If an over-allocated job has already produced what you need, cancel it rather than letting the wallclock run out.

Three other patterns matter. Do not hog: requesting 64 CPUs and 256 GB when the job needs 4 CPUs and 16 GB makes your job harder to schedule and blocks others. Do not trash GPFS metadata: workflows that write millions of tiny files slow the filesystem for everyone; use Parquet and tarballs instead. Do not get the cluster IP-blocked: every job leaves the network from the same outbound IP, so an aggressive scraper or API client can break downloads for everyone.

For collaborative work, email `somit@yale.edu` for a shared `/gpfs/project/<name>/` folder instead of scattering files across personal directories. Shared folders carry proper permissions, reduce duplication, and survive when one team member's account is offboarded.

Working with Claude Code on the HPC

This guide has a companion: the Yale SOM HPC marketplace (`yale-som-hpc/claude-code-marketplace`). It is a Claude Code plugin that ships skills — instructions written for the AI — matching the sections of this document. When you ask Claude Code to work on the cluster, it loads the relevant skill and follows the conventions encoded there.

The two artifacts do different jobs. This document teaches you the cluster's mental model and operating rules, so you can work without Claude and judge Claude's plans when you use it. The marketplace teaches the AI those same rules, so "submit this as a Slurm job" includes thread exports, partition selection, atomic writes, GPU etiquette, and the other details that matter. Neither replaces the other. A human who does not understand the cluster cannot evaluate the AI's output; an AI without the marketplace can produce plausible instructions that violate local conventions.

Table 3 maps common requests to the skill Claude Code will load.

"I want to…"	*Skill*
Get oriented on the cluster	`overview`
Set up SSH keys, Jupyter tunnels	`connecting-securely`
Submit, array, or chain Slurm jobs	`managing-jobs`
Check whether my last job was wasteful	`self-diagnosing-resource-use`
Choose where to store files on GPFS	`using-the-filesystem`
Install software / fix a GLIBC error	`installing-software`
Create a new project layout	`starting-a-new-project`
Run a Python job	`running-python`
Parallelize Python work	`parallel-python`
Speed up slow Python	`accelerating-python`
Run an R job	`running-r`
Run a Stata job	`running-stata`
Request a GPU	`using-gpus`
Process larger-than-memory data	`working-with-large-data`
Download data, call APIs, query WRDS	`acquiring-data`

Table 3: Map from research task to the skill Claude Code will load.

For current install instructions, see the marketplace `README.md`. The short version, from inside Claude Code, is:

```
/plugin marketplace add yale-som-hpc/claude-code-marketplace
/plugin install hpc@yale-som-hpc
```

Getting help

Keep a short command list nearby. For jobs running now, use `squeue --me`. For pending jobs, use `squeue --me --start` and `scontrol show job <id>`. For post-run resource use, use `seff <id>` and `sacct -j <id>`. For partition and node details, use `sinfo -s`. For account, storage, and project-folder requests, email `somit@yale.edu`.

Reference documentation lives at slurm.schedmd.com for Slurm and lmod.readthedocs.io for Lmod. The marketplace skills are a second source for the operational patterns in this guide.

Appendix: example files

These files form a complete first job: a Python script, a Slurm submission script, and the commands to submit and inspect the run. Copy them verbatim, run them end to end, and then modify them for your own work. It is a toy job, but it walks through the same door as the real ones.

fibonacci.py

```python
#!/usr/bin/env python3
"""
Compute Fibonacci numbers. A trivial Python script for testing
HPC submission end-to-end.
"""

import sys
import time

def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def fibonacci_sequence(n):
    return [fibonacci(i) for i in range(n)]

if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    print(f"Computing first {n} Fibonacci numbers...")
    t0 = time.time()
    seq = fibonacci_sequence(n)
    elapsed = time.time() - t0
    for i, fib in enumerate(seq):
        print(f"F({i}) = {fib}")
    print(f"\nDone in {elapsed:.4f} s. F({n}) = {fibonacci(n)}")
```

submit_fibonacci.sh

```bash
#!/bin/bash
#SBATCH --job-name=fibonacci
#SBATCH --partition=default_queue
#SBATCH --output=logs/fibonacci_%j.out
#SBATCH --error=logs/fibonacci_%j.err
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
```

```
export PYTHONUNBUFFERED=1

# Print job information
echo "Job started at: $(date)"
echo "Running on node: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "==========================================="

# Load Python and run the script.
module load python
srun python fibonacci.py 30

echo "==========================================="
echo "Job finished at: $(date)"
```

Running it

```
mkdir -p logs
sbatch submit_fibonacci.sh
squeue --me
# when the job finishes:
seff <job-id>
cat logs/fibonacci_<job-id>.out
```

The `seff` call at the end is the habit to build from day one.